

The EEQ Charge-Equilibration Calculator

Generated/Reviewed Documentation (Codex/Opus/Human) for v0.0.1

1 What the calculator computes

The EEQ calculator assigns a geometry-dependent partial charge q_i to each atom i in a protein conformation. It uses the extended electronegativity equalization model: atoms exchange continuous charge until the marginal energy cost of putting a little more charge on any atom is the same, while the total charge of the conformation is held fixed. In this implementation the default total charge is $Q = 0$ e, so the stored charge vector is neutral unless the later per-atom clamp is triggered.

These charges bear on shielding through electric-field effects. A distribution of charge produces an electric field $\mathbf{E}$ and an electric-field gradient $\mathbf{V}$ at a nucleus. In Buckingham's expansion, the electronic shielding can change with the local electric field, and the anisotropic part can couple to the rank-2 field-gradient tensor. That is the physical route from a charge model to an NMR shielding descriptor.

The code for `EeqResult` stops before the field calculation. It writes `eeq_charges.npy`, an N -component vector of charges in elementary charge units, and `eeq_cn.npy`, the coordination numbers used while building those charges. It does not emit a shielding tensor, an electric field, an electric-field gradient, or a polarizability. The output is therefore a geometry-derived charge descriptor, not a chemical shift and not a shielding in ppm.

2 Where shielding enters

For a nucleus in an applied magnetic field $\mathbf{B}_0$, the local field is usually written

$$\mathbf{B}_{\text{nuc}} = (\mathbf{I} - \boldsymbol{\sigma})\mathbf{B}_0. \quad (1)$$

$\mathbf{I}$ is the 3×3 identity matrix, and $\boldsymbol{\sigma}$ is the nuclear shielding tensor. In physical NMR $\boldsymbol{\sigma}$ is dimensionless: its entry σ_{ab} says how a field applied along Cartesian direction b changes the field component felt by the nucleus along direction a . The tensor form is needed because a molecule has orientation. Rotating the same local structure can change both the magnitude and the direction of the secondary magnetic field created by the electrons.

The quantum-mechanical origin is electronic response. The magnetic field perturbs the electronic state; the perturbed electrons create their own field at the nucleus; Eq. (1) records the linear coefficient between the applied field and the field that reaches the nucleus. Electric fields enter because the electron cloud is also shaped by the electrostatic environment. A nearby charge distribution can pull electron density toward one side of a bond or lone pair, changing the magnetic response of that electronic state.

For point charges q_j , the vacuum field and field gradient at a target atom i would have the Coulomb forms

$$E_a(i) = k_e \sum_{j \neq i} q_j \frac{(r_i - r_j)_a}{|\mathbf{r}_i - \mathbf{r}_j|^3}, \quad (2)$$

$$V_{ab}(i) = k_e \sum_{j \neq i} q_j \left[\frac{3(r_i - r_j)_a(r_i - r_j)_b}{|\mathbf{r}_i - \mathbf{r}_j|^5} - \frac{\delta_{ab}}{|\mathbf{r}_i - \mathbf{r}_j|^3} \right]. \quad (3)$$

Here $\mathbf{r}_i$ is the target position, $\mathbf{r}_j$ is a source-charge position, $a, b \in \{x, y, z\}$, δ_{ab} is one when $a = b$ and zero otherwise, and $k_e = 14.3996 \text{ V } \text{\AA}$ converts the raw $\text{e}/\text{\AA}^2$ and $\text{e}/\text{\AA}^3$ sums to $\text{V}/\text{\AA}$ and $\text{V}/\text{\AA}^2$. Equations (2) and (3) explain why a charge model matters for shielding, but they are not executed by `EeqResult.cpp`.

3 The tensor split used by the pipeline

When a calculator does produce a 3×3 tensor $\mathbf{X}$, the shared `SphericalTensor` code decomposes it into scalar, antisymmetric, and symmetric-traceless pieces. The scalar part is

$$T_0 = \frac{1}{3} \text{tr } \mathbf{X}. \quad (4)$$

This is the isotropic value: it multiplies the identity matrix and is the part that survives orientational tumbling in solution.

The antisymmetric part is stored as a three-component pseudovector,

$$\mathbf{T}_1 = \frac{1}{2} \begin{pmatrix} X_{yz} - X_{zy} \\ X_{zx} - X_{xz} \\ X_{xy} - X_{yx} \end{pmatrix}. \quad (5)$$

This is a compact representation of the part of $\mathbf{X}$ that changes sign under transpose. It is usually not observed directly in standard isotropic solution NMR.

The remaining part is the symmetric traceless matrix

$$\mathbf{S} = \frac{\mathbf{X} + \mathbf{X}^T}{2} - T_0 \mathbf{I}, \quad \text{tr } \mathbf{S} = 0. \quad (6)$$

$\mathbf{S}$ has five independent entries. The code packs them as

$$\mathbf{T}_2 = \begin{pmatrix} \sqrt{2} S_{xy} \\ \sqrt{2} S_{yz} \\ \sqrt{3/2} S_{zz} \\ \sqrt{2} S_{xz} \\ (S_{xx} - S_{yy})/\sqrt{2} \end{pmatrix}, \quad (7)$$

in the order $m = -2, -1, 0, +1, +2$. These factors are the normalization in `Types.cpp`; they preserve the Frobenius norm, so $\sum_m T_{2m}^2 = \sum_{ab} S_{ab}^2$. The packed nine-value layout is

$$[T_0, T_1[0], T_1[1], T_1[2], T_2[0], T_2[1], T_2[2], T_2[3], T_2[4]].$$

EEQ does not populate this tensor layout. A partial charge is a scalar attached to an atom. It can be treated as a rotational rank-0 feature in a later model, but it is not the T_0 part of a shielding tensor. The anisotropic rank-2 argument enters when charges are turned into a field-gradient tensor such as Eq. (3); the present EEQ source file does not do that step.

4 The method implemented

The implementation first reads atom elements and coordinates from the conformation. Coordinates are converted from Å to a_0 using

$$1 a_0 = 0.52917721067 \text{ Å}.$$

For each element it reads a project-local D4 parameter row from `PhysicalConstants.h`: electronegativity χ , hardness η (named `gam` in the code), a coordination-number electronegativity shift κ , covalent radius r_{cov} , and Gaussian charge width r_g . The values used for protein elements are:

Elem.	χ / Ha	η / Ha	κ / Ha	r_{cov} / a_0	r_g / a_0
H	-0.35015861	0.47259288	-0.19793756	0.80628	1.61478
C	-0.04726052	0.25364654	0.14216971	1.51718	2.49988
N	0.11527249	0.28022740	0.15169154	1.42165	2.23456
O	0.25136810	0.36515829	0.14510449	1.24854	1.89247
S	0.10789083	0.25140725	0.15916035	2.00000	3.29733

An unrecognized element uses the fallback row (0, 0.30000000, 0, 1.50000, 2.50000) in the same units.

The first computed quantity is a smooth coordination number. For each atom pair within the configured cutoff $R_{\text{cut}} = 25.0 \text{ \AA}$, the code adds

$$c_{ij} = \frac{1}{2} \operatorname{erfc} \left[k_{\text{CN}} \left(\frac{R_{ij}}{r_{\text{cov},i} + r_{\text{cov},j}} - 1 \right) \right], \quad k_{\text{CN}} = 7.5. \quad (8)$$

R_{ij} is the interatomic distance in a_0 . The complementary error function erfc makes the count change smoothly from near one for close bonded neighbours to near zero for separated atoms. The total coordination number is

$$\text{CN}_i = \sum_{j \neq i} c_{ij}. \quad (9)$$

Pairs beyond R_{cut} are skipped.

The coordination number shifts the electronegativity:

$$\chi_i^{\text{eff}} = \chi_i + \kappa_i \sqrt{\text{CN}_i + 10^{-14}}. \quad (10)$$

The small 10^{-14} term is a guard against evaluating the square root at zero. χ_i^{eff} has units of Ha per elementary charge in atomic-unit bookkeeping.

The charge vector is then obtained by minimizing a quadratic energy subject to a total-charge constraint:

$$E(\mathbf{q}) = \sum_i \chi_i^{\text{eff}} q_i + \frac{1}{2} \sum_i A_{ii} q_i^2 + \sum_{i < j} A_{ij} q_i q_j, \quad \sum_i q_i = Q. \quad (11)$$

q_i is measured in elementary charges, and Q is the configured total charge, defaulting to 0 e. The matrix diagonal is

$$A_{ii} = \eta_i + \frac{\sqrt{2/\pi}}{r_{\text{g},i}}, \quad (12)$$

where the second term is the self-Coulomb energy of the Gaussian charge distribution used by the model. For $i \neq j$, the implementation uses the Ohno–Klopman form

$$A_{ij} = \gamma_{ij}(R_{ij}) = \frac{1}{\sqrt{R_{ij}^2 + 1/(\eta_i \eta_j)}}. \quad (13)$$

This is the code's off-diagonal Coulomb kernel. It differs from the dftd4 reference implementation, which uses a Gaussian-charge $\operatorname{erf}(\alpha R)/R$ form. At long range Eq. (13) approaches the atomic-unit Coulomb factor $1/R_{ij}$. At short range it remains finite, approaching $\sqrt{\eta_i \eta_j}$ as $R_{ij} \rightarrow 0$.

The equalization condition follows by differentiating Eq. (11). Define the marginal charge energy

$$\mu_i(\mathbf{q}) = \frac{\partial E}{\partial q_i} = \chi_i^{\text{eff}} + \sum_j A_{ij} q_j. \quad (14)$$

Moving a small amount of charge from atom j to atom i changes the energy to first order by $(\mu_i - \mu_j) dq$. At the constrained minimum those exchange costs must match, so the code solves

$$\begin{bmatrix} \mathbf{A} & \mathbf{1} \\ \mathbf{1}^\top & 0 \end{bmatrix} \begin{bmatrix} \mathbf{q} \\ \lambda \end{bmatrix} = \begin{bmatrix} -\boldsymbol{\chi}^{\text{eff}} \\ Q \end{bmatrix}. \quad (15)$$

$\mathbf{1}$ is the vector of ones, and the Lagrange multiplier λ is the common offset that enforces the total charge. The augmented system is the algebraic form of equal marginal energy plus charge conservation.

Rather than factor the bordered matrix in Eq. (15), the code factors $\mathbf{A}$ with Eigen's Cholesky routine LLT. Cholesky writes a symmetric positive-definite matrix as $\mathbf{L}\mathbf{L}^\top$, turning later linear solves into triangular substitutions. It then solves

$$\mathbf{A}\mathbf{u} = \boldsymbol{\chi}^{\text{eff}}, \quad \mathbf{A}\mathbf{v} = \mathbf{1}, \quad (16)$$

and computes

$$\lambda = -\frac{Q + \mathbf{1}^\top \mathbf{u}}{\mathbf{1}^\top \mathbf{v}}, \quad \mathbf{q} = -(\mathbf{u} + \lambda \mathbf{v}). \quad (17)$$

If the factorization fails, or if the denominator in Eq. (17) is smaller than 10^{-30} , the result is not attached.

After the solve, the code removes the floating-point charge residual by a uniform shift:

$$q_i \leftarrow q_i - \frac{\sum_j q_j - Q}{N}. \quad (18)$$

This preserves charge differences while making the pre-clamp sum equal to Q to roundoff. A guard then clamps any stored atom charge with $|q_i| > 2.0\text{e}$ to $\pm 2.0\text{e}$. If that guard fires, the stored charge sum can move away from Q . The code records clamp events and charge statistics for audit output.

5 Inputs and output

EEQ consumes atom elements, atom positions in Å, and four TOML configuration values: $Q = 0\text{e}$, $k_{\text{CN}} = 7.5$, $R_{\text{cut}} = 25.0\text{Å}$, and the 2.0e charge clamp. It has no declared `ConformationResult` dependency and no external binary.

It does not consume MOPAC charges, AIMNet2 charges or embeddings, ff14SB charges, APBS electrostatic fields, or DSSP secondary-structure labels. Those data sources feed other parts of the pipeline: MOPAC and AIMNet2 provide separate charge models, ff14SB charges feed the force-field Coulomb field, APBS provides a solvated field and field gradient, and DSSP supplies secondary structure labels. In the current code, no field calculator reads `eeq_charge`.

The feature output is scalar. `eeq_charges.npy` has shape $(N,)$ and stores q_i in elementary charges. `eeq_cn.npy` has shape $(N,)$ and stores CN_i . Across a trajectory, `EeqWelfordTrajectoryResult` accumulates means, standard deviations, minima, maxima, frame-to-frame changes, and charge-change rates for q_i . None of these values is a shielding. Calibration or later modelling is the step that can test whether this geometry-responsive charge signal correlates with DFT shielding changes.